

BASH KOMUT DOSYASI YAZIMI

[Kaynak](#)

İÇERİK

- Bash Programlamaya Giriş ve Temel Yapı
- Bash Programlamada Değişkenler ve Hata Ayıklama
- Parametreler ve İnteraktif Girdi (Input)
- Bash Dizileri (Arrays)
- Koşullu İfadeler (If/Else)

Bash Programlamaya Giriş ve Temel Yapı

Bash script yazımı, tekrarlanan terminal komutlarını otomatize etmek ve sistem yönetimini kolaylaştırmak için kullanılan güçlü bir yöntemdir. Aşağıda, bir Bash scriptinin temel yapı taşları ve çalışma mantığı detaylandırılmıştır.

1. Script Yapısı ve Shebang Satırı

Bir Bash scripti her zaman dosyanın en üstünde yer alan özel bir kod satırı ile başlar. Bu satıra **Shebang** adı verilir:

```
#!/bin/bash
```

- Neden Önemlidir?:** Bu satır, kullandığınız shell (kabuk) türü ne olursa olsun, işletim sistemine bu dosyanın içerisindeki komutları **bash** yorumlayıcısını kullanarak çalıştırması gerektiğini bildirir.
- Kullanım Gerekliliği:** Eğer bu satırı eklemeszeniz, sistem dosyayı varsayılan kabuk ile çalıştırmaya çalışabilir ve Bash'e özgü komutlarda hatalar meydana gelebilir.

2. Temel Komutlar ve Çıktı İşlemleri

Bash içerisinde hem Bash'e özgü komutlar hem de standart Linux komutları birlikte kullanılabilir.

- echo Komutu:** Ekranı metin yazdırmak için kullanılır. Python dilindeki `print` fonksiyonu ile aynı işlevi görür.
 - Örnek:** `echo "Hello World"` komutu terminale "Hello World" metnini döndürür.
- Linux Komutlarının Entegrasyonu:** Script içerisinde standart Linux komutlarını (ls, whoami, id vb.) doğrudan kullanabilirsiniz. Format doğru olduğu sürece script bu komutları sırasıyla icra edecektir.

- **Örnek Senaryo:** Bir script içerisine önce `whoami` (mevcut kullanıcıyı gösterir) ve ardından `id` (kullanıcı ve grup bilgilerini gösterir) komutlarını yazarsanız, script çalıştığında bu iki bilginin çıktısını alt alta verir.

3. Script Dosyasını Hazırlama ve Çalıştırma

Bir metin dosyasına komutları yazmak, onu çalıştırmak için yeterli değildir. Dosyanın sistem tarafından "çalıştırılabilir" olarak tanınması gerekir.

A. Yetkilendirme (Permission)

Dosyayı oluşturduktan sonra, terminal üzerinden **executable (çalıştırılabilir)** izni vermeniz gerekir. Bunun için `chmod` komutu kullanılır:

```
chmod +x dosya_adiniz.sh
```

Bu komut, dosyaya çalıştırma yetkisi ekleyerek sistemin onu bir program gibi görmesini sağlar.

B. Çalıştırma Metodu

Yetkilendirme yapıldıktan sonra, scripti bulduğunuz dizinden şu formatla çalıştırabilirsiniz:

```
./dosya_adiniz.sh
```

4. Uygulama Örneği

Aşağıda, yukarıdaki tüm kuralları kapsayan basit bir Bash script örneği yer almaktadır:

Bash

```
#!/bin/bash

# Ekrana karşılama metni basar
echo "Sistem Bilgileri Getiriliyor..."

# Mevcut dizindeki dosyaları listeler
ls

# Mevcut kullanıcı adını ve ID bilgilerini yazdırır
whoami
id
```

Bash Programlamada Değişkenler ve Hata Ayıklama

Bu bölüm, verilerin nasıl depolanacağını, bu verilere nasıl erişileceğini ve yazılan scriptlerdeki hataların nasıl tespit edileceğini kapsamaktadır.

1. Değişken Tanımlama ve Kullanımı

Bash'te değişkenler, verileri saklamak ve kodun farklı yerlerinde tekrar kullanmak için oluşturulan isimlendirilmiş birimlerdir.

A. Değişken Atama Kuralları

Değişken oluştururken uyulması gereken **katı kurallar** vardır:

- **Boşluk Bırakılmamalıdır:** Değişken adı, eşittir sembolü (=) ve atanan değer arasında kesinlikle boşluk olmamalıdır.
 - **Doğru:** `name="Jammy"`
 - **Yanlış:** `name = "Jammy"` (Bash bunu bir komut olarak algılar ve hata verir).
- **Veri Atama:** Değerler çift tırnak içerisinde yazılabilir, bu özellikle boşluk içeren metinler için önemlidir.

B. Değişkene Erişme (Çağırma)

Bir değişkenin değerini kullanmak için değişken adının başına dolar işareti (\$) getirilmelidir.

- **Örnek:** `echo $name` komutu, ekrana değişkenin içeriği olan "Jammy" ifadesini yazdırır.
- **Çoklu Kullanım:** Bir satırda (örneğin bir `echo` ifadesinde) birden fazla değişken aynı anda kullanılabilir.
 - **Senaryo:** `name="Jammy"` ve `age="25"` değişkenleriniz varsa, `echo $name $age` komutuyla her iki veriyi de yan yana yazdırabilirsiniz.

C. Değişken Kullanmanın Avantajı

Değişkenler, aynı veriyi kodun onca farklı yerinde elle yazmak yerine tek bir merkezden yönetmenizi sağlar. Eğer veriyi değiştirmeniz gerekirse, sadece değişkenin tanımlandığı satırı güncellemeniz yeterli olacaktır; bu da hata payını düşürür.

2. Hata Ayıklama (Debugging)

Programlama sürecinde hataları çözmek (problem solving), geliştirmenin en önemli parçasıdır. Bash, hataların yerini tespit etmek için yerleşik özellikler sunar.

A. Terminal Üzerinden Debug Modu

Bir scripti hata ayıklama modunda çalıştırmak için script isminin başına `-x` parametresi eklenir:

```
bash -x ./file.sh
```

- **Çalışma Mantığı:** Bu modda Bash, çalıştırdığı her satırı terminale basar.

- **Göstergeler:**
 - **Artı (+) İşareti:** Komutun başarıyla işlendiğini ve o satırın yürütüldüğünü gösterir. Ardından komutun çıktısı gelir.
 - **Eksi (-) İşareti:** Bir hata oluştuğunu gösterir. Bu sayede scriptin hangi satırda kilitlendiğini veya yanlış sonuç verdiğini anında görebilirsiniz.

B. Kod İçerisinde Bölgesel Debugging

Tüm scripti değil de sadece belirli bir bölümü incelemek istiyorsanız scriptin içine şu komutları yerleştirebilirsiniz:

- `set -x` : Hata ayıklama modunu bu satırdan itibaren **başlatır**.
- `set +x` : Hata ayıklama modunu bu satırda **durdurur**.

Bu yöntem, özellikle yüzlerce satırlık büyük scriptlerde sadece şüpheli görülen kod bloklarını izole etmek için kritiktir.

3. Örnek Senaryo: Hatalı Kod Analizi

Aşağıdaki basit scripti düşünelim:

Bash

```
#!/bin/bash
name="Jammy"
echo "Merhaba $name"
# Hata yapalım:
ls -z # Geçersiz bir parametre
```

Bu scripti `bash -x ./test.sh` ile çalıştırdığınızda:

1. `+ name=Jammy` satırını görürsünüz (Atama başarılı).
2. `+ echo 'Merhaba Jammy'` ve çıktısını görürsünüz.
3. `ls -z` satırında bir hata mesajı ve yanında işaretleyici belirir, böylece sorunun `ls` komutuna verilen `-z` parametresinden kaynaklandığını anlarsınız.

Parametreler ve İnteraktif Girdi (Input)

Bash scriptlerinde dışarıdan veri almanın iki temel yolu vardır: Scripti çalıştırırken argüman göndermek veya script çalışırken kullanıcıdan girdi istemek.

1. Komut Satırı Argümanları (Konumsal Parametreler)

Bir scripti terminalde çalıştırırken yanına yazdığınız her kelime bir **parametre** olarak kabul edilir. Bu parametreler script içinde özel değişken isimleri olan `$1` , `$2` , `$3` ... şeklinde

saklanır.

A. Birinci Argümanı Yakalama

Scriptinizi çalıştırırken verdiğiniz ilk değeri yakalamak için `$1` kullanılır.

- **Kod:** `name=$1`
- **Çalıştırma:** `./example.sh Alex`
- **Sonuç:** `$1` değişkeni "Alex" değerini alır ve ekrana basıldığında "Alex" çıktısı verir.

B. Çoklu Argüman Kullanımı

Sıralama arttıkça rakam da artar.

- **Senaryo:** İkinci bir ismi yakalamak istiyorsanız `$2` kullanırsınız.
- **Kod:** `name2=$2`
- **Çalıştırma:** `./example.sh Alex Tony`
- **Sonuç:** Bu durumda `$1` "Alex", `$2` ise "Tony" değerini taşır.

Önemli Not: Parametreler de özünde birer değişken olduğu için çağrılırken mutlaka `$` öneki kullanılmalıdır.

2. İnteraktif Girdi: `read` Komutu

Bazen verileri scripti çalıştırmadan önce değil, script çalışırken kullanıcıya sorarak almak isteyebilirsiniz. Bu süreci **interaktif** hale getiren komut `read` komutudur.

- **Çalışma Mantığı:** Script `read` satırına geldiğinde duraklar (hang) ve kullanıcının klavyeden bir metin girip "Enter" tuşuna basmasını bekler.
- **Örnek Kullanım:**

Bash

```
echo "Lütfen adınızı giriniz:"
read name
echo "Merhaba $name, sisteme hoş geldin!"
```

- **Sonuç:** Kullanıcı ismini yazar yazmaz, girilen bu veri `name` değişkenine atanır ve bir sonraki satırda kullanılır.

3. Uygulama ve Alıştırma: Biyografi Oluşturucu

Öğrenilenleri pekiştirmek için en iyi yöntem, birden fazla parametreyi birleştiren bir senaryo üzerinde çalışmaktır.

Senaryo Ödevi:

Bir "Biyografi Oluşturucu" scripti yazın. Bu script:

- Kullanıcının **adını**, **yaşını** ve **mesleğini** parametre veya `read` yoluyla alsın.
- Bu bilgileri değişkenlerde depolasın.
- Sonuç olarak anlamlı bir cümle kurarak ekrana yazdırsın.
 - Örnek Çıktı:** "Merhaba, benim adım Alex. 25 yaşındayım ve Siber Güvenlik Uzmanı olarak çalışıyorum."

Parametre Türleri Özeti (Tablo)

Parametre	Tanımı
\$0	Scriptin kendi adını tutar.
\$1 - \$9	Scriptin yanına yazılan 1. ile 9. arasındaki argümanları tutar.
\$#	Scriptin yanına toplam kaç adet argüman yazıldığını gösterir.
\$@	Verilen tüm argümanları bir liste olarak tutar.

Bash Dizileri (Arrays)

Diziler, bir değişken içinde birden fazla veri parçasını depolamak için kullanılır. Depolanan her bir veriye bir **indeks (index)** numarası atanır ve bu numara üzerinden veriye erişim sağlanır.

1. İndeksleme Mantığı (Zero-Indexing)

Bash ve çoğu programlama dilinde diziler **0 (sıfır)** ile başlar. Bu, dizideki ilk öğenin numarasının 1 değil, 0 olduğu anlamına gelir.

Örnek Senaryo: Taşıtlar Dizisi

Dizimiz: `['car', 'train', 'bike', 'bus']` olsun.

Öge (Item)	İndeks Numarası
car	0
train	1
bike	2
bus	3

2. Dizi Tanımlama ve Yazdırma

Bash'te bir dizi tanımlarken parantez `()` kullanılır ve her öge arasında bir boşluk bırakılır.

A. Dizi Oluřturma

```
transport=('car' 'train' 'bike' 'bus')
```

B. Tüm Elemanları Yazdırma

Dizideki tüm öğeleri bir kerede ekrana basmak için @ sembolü kullanılır:

```
echo "${transport[@]}"
```

- **Açıklama:** Burada @ tüm argümanları, [] ise indeks alanını temsil eder.

C. Belirli Bir Elemana Eriřme

Sadece belirli bir sıradaki öğeyi çekmek için o öğenin indeks numarasını köşeli parantez içine yazmalısınız:

```
echo "${transport[1]}"
```

- **Sonuç:** Ekrana train yazdırılır (çünkü 1. indekste tren bulunmaktadır).

3. Diziyi Güncelleme ve Silme

Diziler dinamik yapılardır; script çalışırken bir elemanı silebilir veya değerini değiřtirebilirsiniz.

- **Eleman Silme (unset):** Diziden bir öğeyi tamamen kaldırmak için unset komutu kullanılır.
 - *Kullanım:* unset transport[1] (Bu komut "train" öğesini diziden siler).
- **Eleman Değıştirme (Atama):** Mevcut bir indeksteki veriyi yenisiyle değiřtirmek için doğrudan atama yapılır.
 - *Kullanım:* transport[1]='trainride'
 - *Sonuç:* Diziyi tekrar yazdığınızda artık "train" yerine "trainride" ifadesini görürsünüz.

4. Uygulama Örneđi ve Pratik

Ařağıdaki dizi üzerinden sistemin nasıl çalıştığını analiz edelim:

```
cars=('honda' 'audi' 'bmw' 'tesla')
```

- **Soru 1:** echo "\${cars[2]}" komutunun çıktısı ne olur?
 - *Cevap:* **bmw** (0: honda, 1: audi, 2: bmw).
- **Soru 2:** Tesla'yı diziden silmek için hangi komutu kullanmalısınız?
 - *Cevap:* unset cars[3]

Geliřtirme Ödevi: Biyografi Projesi (V2)

Önceki derste yaptığınız "Biyografi Oluşturucu" scriptine dizileri entegre edin.

- Kullanıcının hobilerini bir dizi (`hobbies`) içinde saklayın.
- Kullanıcıya ait 3 farklı gerçeği bir dizide tutun ve bu dizinin sadece 2. elemanını (indeks 1) ekrana bastıran bir bölüm ekleyin.

Koşullu İfadeler (If/Else)

Bir kod bloğunun çalışması bir şarta bağlı olduğunda "if" (eğer) yapısı kullanılır. Bu şartlar genellikle **ilişkisel operatörler** (eşitlik, büyüklük, küçüklük) ile kontrol edilir.

1. Temel "If" Yapısı ve Yazım Kuralları

Bash'te `if` ifadeleri çok katı yazım kurallarına (syntax) sahiptir. En küçük hata scriptin bozulmasına neden olur.

Genel Yapı:

Bash

```
if [ condition ]
then
    # Şart doğruysa çalışacak kodlar
else
    # Şart yanlışsa çalışacak kodlar
fi
```

Kritik Yazım Kuralları:

- Boşluklar:** Köşeli parantezlerin `[` iç kısmında, her iki yanda da mutlaka **birer boşluk** bırakılmalıdır. (`[ $a -eq $b ]` gibi).
- Kapanış:** Her `if` bloğu mutlaka `fi` (if'in tersi) kelimesi ile sonlandırılmalıdır.

2. Karşılaştırma Operatörleri

Sayısal veya metinsel karşılaştırmalar için kullanılan temel operatörler şunlardır:

Operatör	Açıklama
<code>-eq(=)</code>	Eşittir (Equal). İki değer birbirine eşitse true döner.
<code>-ne(!=)</code>	Eşit Değildir (Not Equal). Değerler farklıysa true döner.
<code>-gt(>)</code>	Büyüktür (Greater Than). Soldaki sağdakinden büyükse true döner.
<code>-lt(<)</code>	Küçüktür (Less Than). Soldaki sağdakinden küçükse true döner.

Operatör	Açıklama
-ge(>=)	Büyük veya Eşittir (Greater or Equal).

3. Dosya Kontrol Operatörleri

Bash, sistem yönetimi için kullanıldığından dosya durumlarını kontrol etmek çok yaygındır:

- **-f**: Dosyanın mevcut olup olmadığını (sıradan bir dosya mı) kontrol eder.
- **-w**: Dosyanın **yazılabilir (writable)** olup olmadığını kontrol eder.

4. Örnek Senaryolar

Senaryo A: Basit Sayı Karşılaştırma (Guessing Game)

Kullanıcının girdiği parametreyi belirli bir değerle karşılaştıran script:

Bash

```
#!/bin/bash
value="guessme"
guess=$1

if [ "$value" = "$guess" ]
then
    echo "Tebrikler, bildiniz!"
else
    echo "Maalesef, yanlış tahmin."
fi
```

Senaryo B: Dosya Varlık ve Yetki Kontrolü

Bir dosyanın varlığını ve yazım iznini kontrol eden, izin yoksa dosyayı yeniden oluşturan script:

Bash

```
#!/bin/bash
file=$1

if [ -f "$file" ] && [ -w "$file" ]
then
    echo "Dosya mevcut ve yazılabilir. Mesaj ekleniyor..."
    echo "hello" > "$file"
else
    echo "Dosya yok veya yetki yetersiz. Yeni dosya oluşturuluyor."
    touch "$file"
```

```
echo "hello" > "$file"
fi
```

Bu örnekte `cat hello.txt` komutuyla dosya içeriğinin "hello" olduğunu teyit edebilirsiniz.

5. Uygulama ve Proje Geliştirme

Daha önce başladığınız "Biyografi Oluşturucu" projesini `if/else` yapısıyla bir üst seviyeye taşıyın:

- Yaş Kontrolü:** Kullanıcının yaşını kontrol edin.
 - Eğer yaş **18'den küçükse**: "Henüz çalışma yaşında değilsiniz" mesajını basın.
 - Eğer yaş **18 veya daha büyükse**: `read` komutuyla kullanıcıya mesleğini sorun ve bilgileri ekrana yazdırın.
 - Kombinasyon:** Hem dizileri hem de parametreleri kullanarak scripti mümkün olduğunca dinamik hale getirin.
-